

FedPDR: Server-Side Post-Plateau Refinement for Federated Learning with Latent Diffusion

Abstract—Federated Learning (FL) has gained prominence for its ability to enable collaborative model training while preserving data privacy in distributed environments. Despite its potential, two critical challenges persist. First, non-Independent and Identically Distributed (non-IID) data across clients induce severe parameter divergence, which conventional linear aggregation fails to reconcile, leading to suboptimal convergence. Second, FL often encounters a performance plateau where frequent client-server synchronization yields diminishing returns. While existing adaptive communication schemes attempt to mitigate this, they often struggle to balance the optimal model accuracy with the stringent resource constraints of edge devices. To address these challenges, we propose FedPDR, a two-stage server-side optimization framework designed to improve both efficiency and convergence in FL. An adaptive plateau detection mechanism is employed to determine the onset of the post-optimization stage. Once a performance plateau is identified, the refinement tasks are offloaded to the server, eliminating further client involvement and mitigating post-plateau inefficiencies. Central to this approach is the use of a latent diffusion model to capture the distribution of global model parameters and generate optimized updates, thereby alleviating suboptimal convergence caused by data heterogeneity. Extensive experiments across multiple benchmarks and real-world settings demonstrate that FedPDR effectively shifts computational burdens, reduces client-side overhead, and consistently improves global model performance while maintaining broad compatibility with existing FL frameworks.

Index Terms—Federated Learning, Model Parameter Generation, Latent Diffusion Model, Post-Training Optimization

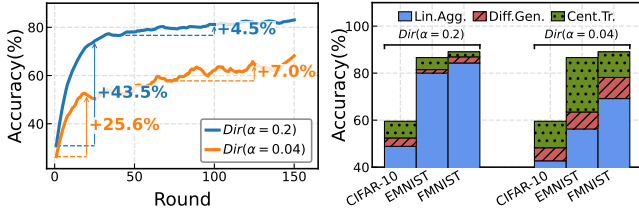
I. INTRODUCTION

The proliferation of intelligent edge devices and IoT systems has led to unprecedented growth of decentralized sensory data, accompanied by increasing on-device computing power and ubiquitous wireless connectivity. In this context, Federated Learning (FL) enables multiple distributed clients, such as smartphones, drones, and industrial sensors, to collaboratively train a shared global model under the coordination of a central server while retaining raw data locally. This paradigm has become a key enabler for privacy-preserving and data-intensive applications, including human activity recognition [1]–[3], health monitoring [4], and industrial analytics [5]. Despite its promise, a key challenge arises from the heterogeneous nature of client data. Due to diverse user behaviors, environments, and usage patterns, local datasets across clients are typically non-Independent and Identically Distributed (non-IID), resulting in inconsistent update directions during training [6]. Such *data heterogeneity* often induces parameter divergence across clients, hindering global model convergence and negatively affecting end-to-end QoS in FL systems [7].

Beyond the challenges posed by non-IID data, FL also commonly experiences a *performance plateau*, where improvements in the global model slow or stagnate after reaching a certain accuracy level. This behavior aligns with the concept of a critical learning period [8], in which the model is highly receptive to gradient information during early training and can quickly incorporate effective client updates to achieve rapid gains. Once this period passes, the model enters a post-plateau stage, becoming progressively less responsive to new updates, as demonstrated by our motivational experiment in Fig. 1(a). While servers generally have ample computational and energy resources, edge clients are constrained by limited processing power and communication bandwidth, making continued FL training increasingly inefficient. Additional communication rounds yield only marginal accuracy gains while exhausting limited client resources, underscoring the need to transfer the optimization workload to the server. Such a shift reduces redundant computation and communication and helps maintain FL performance in resource-constrained environments.

Existing research has extensively explored methods to mitigate statistical heterogeneity in FL [9]. For example, FedProx [9] uses a proximal term to stabilize local training and reduce client drift, and SCAFFOLD exploits control variates to lessen the variance of local updates [10]. However, these methods still rely on linear aggregation and do not take into account the differences in parameter distribution caused by heterogeneous client data and the high-dimensional complexity of model parameters, making it difficult to break the performance bottleneck brought by traditional linear aggregation mechanisms. As shown in Fig. 1(b), when data heterogeneity increases, the performance of the model obtained through linear aggregation will decrease. The results of diffusion generation indicates that changing the aggregation method can alleviate the influence of data heterogeneity. Other approaches try to mitigate data heterogeneity by leveraging prototype learning or data generation to construct shared representations across clients. These methods may incur additional computational overhead on the client side and may introduce privacy risks due to the exposure of prototype information or synthetic data [11], [12]. Moreover, few methods explicitly tackle the diminishing returns observed during prolonged FL training, leaving open the question of how to effectively adapt the learning process to the post-plateau stage and highlighting significant opportunities for further improvement.

To overcome these challenges, we propose FedPDR, a server-side post-plateau refinement framework for FL. Specifically, we present an Adaptive Plateau Detection (APD)



(a) Performance Plateau. (b) Data Heterogeneity.

Fig. 1: Illustration of performance plateau and data heterogeneity challenges in conventional FL.

mechanism that monitors model accuracy during training and identifies the onset of the performance plateau by distinguishing effective updates from noisy ones. Once detected, the method enters the post-plateau refinement phase. Rather than treating global aggregation as simple averaging, we model the plateau-stage parameters on the server as a high-dimensional distribution that captures rich structural information. Inspired by the ability of diffusion models to learn complex data manifolds, we employ a Latent Diffusion Model (LDM) to learn this implicit parameter distribution and generate refined global parameters as probabilistic samples, bypassing further client aggregation. By using APD to trigger this process, we convert the inefficient post-plateau phase into a server-side generative optimization, removing redundant computation and communication on clients while continuing to improve model performance. This design of adaptive stage detection, parameter distribution modeling, and server-side refinement allows FedPDR to address both convergence degradation from data heterogeneity and resource inefficiency from extended training. The main contributions are summarized as follows,

- We present the APD mechanism, which dynamically tracks the training trajectory to detect the onset of the performance plateau and optimally trigger server-side task offloading, minimizing client overhead.
- We propose FedPDR, a post-optimization framework that leverages a latent diffusion model to refine the global model from high-performance checkpoints, overcoming the limitations of linear aggregation while reducing computation and communication costs for edge devices.
- Extensive experiments are conducted on multiple benchmark datasets, FL frameworks, and a real-world deployment to evaluate the effectiveness of server-side post-plateau optimization. Results demonstrate that our FedPDR method reduces client computation and communication overhead by an average of 63% while consistently improving global model accuracy by 4.7%.

II. MOTIVATION AND CHALLENGE

To better understand the performance of existing linear aggregation methods and the potential benefits of generative refinement in FL, we conduct a pilot study with 10 clients on Fashion-MNIST under non-IID conditions ($\alpha = 0.2$), using a three-stage CNN as the model backbone.

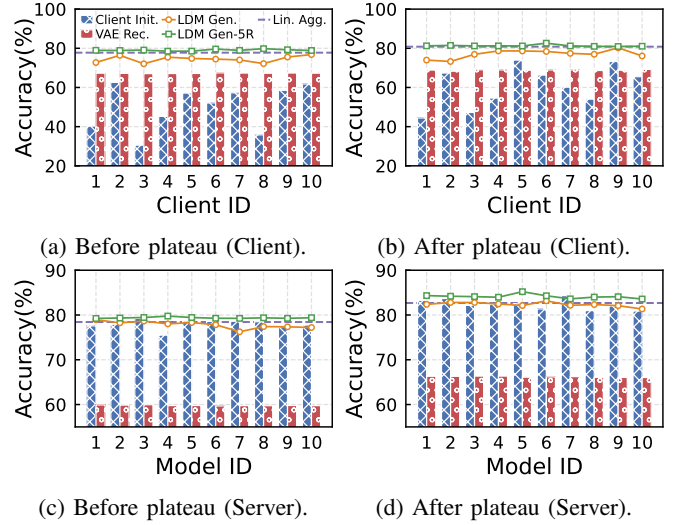


Fig. 2: Models collected at two stages of FL and their performance under different optimization strategies.

We first construct experience pools at distinct stages of FL by collecting two types of model weights: (i) the local models from all participating clients, and (ii) the aggregated global models. We then compare three optimization strategies:

- **Simple Averaging** (Lin. Agg.): Traditional linear averaging of the models in the experience pool [13].
- **VAE Reconstruction** (VAE Rec.): Train a variational autoencoder (VAE) on the experience pool and evaluate the reconstructed models.
- **LDM Generation** (LDM Gen.): Train an LDM on the pool to sample and synthesize refined parameters. LDM Gen-5R represents 5 rounds of iterative generation.

A. VAE as a Latent Aggregator for Model Consensus

Our first observation concerns the averaging effect of the VAE. Despite the high variance in the performance of the initial local models caused by data heterogeneity, the VAE-reconstructed models exhibit highly consistent performance. For example, as shown in Fig. 2(a-b), the models of different clients exhibit significant differences due to non-IID data, but after VAE reconstruction, the performance of the client models tends to be consistent, indicating that the latent space can capture non-linear relationships among model parameters.

B. LDM for Generative Extrapolation Beyond Linear Limits

The second observation highlights the breakthrough potential of the LDM. For example, as shown in Fig. 2(c-d), the parameters generated by the diffusion process consistently reach or exceed the accuracy of the linear aggregation baseline. This indicates that LDM can learn the implicit distribution of high-performance regions in the parameter space. As a probabilistic denoising generative process, LDM can sample and generate better model parameters in high-performance regions of the parameter space. This extrapolation ability beyond the convex hull of existing models enables FedPDR

to overcome the performance bottleneck of linear aggregation and obtain a better global model.

C. Challenges in Resource-Constrained FL

The primary challenge lies in the significant computational asymmetry between servers and resource-constrained edge devices. In real-world scenarios, clients are constrained by computing power and bandwidth. During the performance plateau, the marginal accuracy improvement brought by local updates cannot offset the high resource consumption of these energy-sensitive nodes. Therefore, it is necessary to shift the optimization burden to the server side to ensure model performance and system sustainability.

Motivated by these insights, we propose a server-side optimization framework that shifts the generative process to the resource-rich server. By allowing clients to exit early from the training loop, the stagnant post-plateau phase is transformed into an efficient, server-only generative optimization process.

III. PRELIMINARIES

A. Client-Server Architecture for FL

We define a FL architecture, which is composed of N clients and a central server. Each client i has its own local private dataset $D_i = \{(x_j^i, y_j^i)\}_{j=1}^{|D_i|}$, where (x_j^i, y_j^i) represents the sample j of the client i . FL trains a global model w to achieve optimal performance across all client data distributions,

$$\min_w F(w) = \frac{1}{N} \sum_{i=1}^N F_i(w), \quad (1)$$

where $F_i(w) = \mathbb{E}_{(x_j^i, y_j^i) \sim D_i} \ell(w; (x_j^i, y_j^i))$, and w represents the global model parameters to be optimized. $F_i(w)$ represents the local objective function of the client i and the average loss of the model w on its local data. ℓ is a specific loss function.

In common FL frameworks, each client performs multiple rounds of training on its local dataset and sends the trained model to the server. The server aggregates all models by $\bar{w} = \sum_{i=1}^N \frac{|D_i|}{\sum_{j=1}^N |D_j|} w^i$ and sends them back to the clients.

B. Variational Autoencoder

VAE is a deep generative model based on variational inference [14]. Its core idea is to learn a low-dimensional latent space that captures the data distribution. VAE assumes that each data point x is generated by a latent variable z . VAE consists of an encoder q_ϕ and a decoder p_ψ . The corresponding loss function is defined as

$$\mathcal{L}_{VAE}(\phi, \psi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\psi(x|z)]}_{\text{Reconstruction Loss}} - \underbrace{\beta \cdot D_{KL}(q_\phi(z|x) \parallel p(z))}_{\text{KL-Divergence Regularization}}. \quad (2)$$

The reconstruction loss maintains high-fidelity data recovery, complemented by KL divergence which regularizes the latent space geometry, aligning the learned posterior with a canonical prior to ensure smoothness and interpretability [15].

C. Diffusion Probabilistic Models

The diffusion model gradually adds noise to the data to form a noise distribution, and then performs denoising through a reverse process to generate new data. It involves a forward process and a reverse process [16], [17].

1) *Forward process*: Given a sample $x_0 \sim q(x)$, the forward process gradually adds Gaussian noise over T steps to produce $x_1, x_2, \dots, x_T$. The formula is defined as

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (3)$$

where $\mathcal{N}$ denotes the Gaussian distribution, β_t controls the noise variance at each step t , and I is the identity matrix. Note that for any step t , we can directly sample

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, \quad (4)$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ and $\epsilon_t \sim \mathcal{N}(0, I)$.

2) *Reverse process*: The reverse process is designed to remove noise from the input x_t to reconstruct data. The reverse process learns how to remove noise by training a neural network parameterized by θ ,

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \quad (5)$$

where $\mu_\theta(x_t, t)$ and $\Sigma_\theta(x_t, t)$ are the estimated Gaussian mean and variance of the denoising network output.

The diffusion model minimizes the noise prediction error by training the noise prediction network. The optimization objective is formulated as

$$\theta^D = \arg \min_{\theta} \mathbb{E}_{(X_0, \epsilon, t)} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2], \quad (6)$$

where θ^D is the optimal noise prediction network obtained through training, and c is condition information [17]. By training a neural network to minimize noise prediction errors, the model can gradually denoise from random noise and obtain the distribution of generated data.

IV. FL WITH POST-HOC DIFFUSION REFINEMENT

As illustrated in Fig. 3, FedPDR is a two-stage framework designed to overcome performance bottlenecks in FL. Its core idea is to autonomously detect the plateau phase and achieve further improvements through a generative refinement process. We first introduce a learning dynamics monitor that automatically identifies when the global model reaches a performance plateau. Upon detection, a set of recent models is collected into a server-side experience pool. A latent diffusion model (LDM) is then trained on this pool to perform offline iterative refinement, generating an improved global model without requiring additional client training. In the following, we provide a detailed description of this framework.

A. Experience Pool Construction

The goal of the first stage is to prepare a high-quality elite model set for the subsequent server-side refinement stage by executing the standard FL algorithm process.

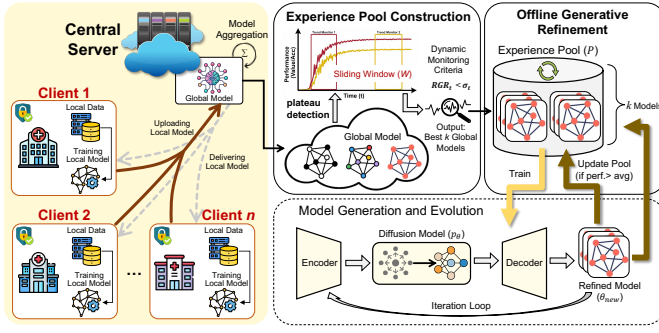


Fig. 3: Framework Overview of our FedPDR.

1) *Experience Pool Collection Startup Conditions*: To identify the transition from the rapid learning phase to the performance plateau, we propose an Adaptive Plateau Detection (APD) mechanism. Specifically, APD models the trajectory of global model accuracy as a composite signal composed of an underlying learning trend and stochastic noise.

We maintain a sliding window of size n , denoted as $H_t = \{\text{acc}_{t-n+1}, \dots, \text{acc}_t\}$. To separate the underlying learning trend from optimization noise, we apply linear regression to H_t using the least squares method and obtain a fitted line $f(i) = k \cdot i + b$, where k is the slope and characterizes the learning rate. Based on this fitted trend, we define the Signal-to-Noise Ratio (SNR) of the learning dynamics as follows,

$$\text{SNR}_t = \frac{\text{Signal}_t}{\text{Noise}_t + \epsilon} = \frac{|k \cdot (n-1)|}{\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - f(i))^2 + \epsilon}}, \quad (7)$$

where y_i is the actual accuracy in the window, $f(i)$ is the predicted value from the linear fit, and ϵ is a small constant to ensure numerical stability.

In this formulation, the numerator represents the cumulative accuracy gain captured by the trend line within the window, while the root mean squared residual quantifies the convergence volatility. A value of $\text{SNR}_t \geq 1$ indicates that the learning signal dominates, whereas $\text{SNR}_t < 1$ signifies that the model has entered a saturation state where noise outweighs the effective gain, and the server triggers the experience pool P to start collecting elite global models.

2) *Update Experience Pool*: When the APD detects that FL training has reached the performance plateau period, we activate the experience pool P and start collecting models. If $|P_t| < K$, then execute the following formula

$$P_{t+1} = P_t \cup \{W_G^t\}, \quad (8)$$

where P is a fixed-size queue with a capacity of K , storing tuples in the form of $(W_i, A(W_i))$. W_i represents model i stored in P , and $A(W_i)$ is the performance of the global model. W_G^t represents the globally aggregated model obtained on the server side in the t -th round of FL. If $|P_t| = K$, stop collecting the model and enter the post-optimization stage.

The pseudocode for the experience pool construction stage is presented in Algorithm 1. The server first initializes the

Algorithm 1: Experience Pool Construction (w/ APD)

Input: Client number N ; Total rounds T ; Local epochs E ; Experience pool size K ; SNR window size $n = 25$; Initial model weights W_0 .

Output: The experience pool P .

```

1 Initialize  $P \leftarrow \emptyset$ ; history queue  $H \leftarrow \emptyset$ ;
2 for each round  $t = 1, 2, \dots, T$  do
3   Server broadcasts  $W_{t-1}$  to all clients;
4   for each client  $i = 1$  to  $N$  in parallel do
5      $W_t^i \leftarrow \text{ClientUpdate}(i, W_{t-1}, E)$ ;
6   Server aggregates:  $W_t \leftarrow \text{Aggregate}(\{W_t^i\})$ ;
7   Evaluate  $W_t$  to obtain accuracy  $A(W_t)$ ;
8    $H.\text{push}(A(W_t))$ ;
9   if  $|H| = n$  then
10    Calculate slope  $k$  and residuals using linear regression on  $H$ ;
11    Compute  $\text{SNR}_t$  according to Eq. (7);
12    if  $\text{SNR}_t < 1$  then
13       $\text{UpdatePool}(P, W_t, K)$  using Eq. (8);
14     $H.\text{pop\_front}()$ ;
15 Return  $P$ ;
```

experience pool and a performance history queue (line 1). In each communication round, the global model is broadcast to clients, locally updated, and then aggregated to obtain the global model W_t (lines 2–6). The performance of W_t is evaluated and recorded in the history queue to monitor recent performance trends (lines 7–8). Once a plateau is detected, the latest K global models W_t are collected into the experience pool according to Eq. 8 (lines 9–14). Finally, the constructed experience pool P is returned (line 15).

B. Offline Generative Refinement

After FL training terminates, we use the high-quality experience pool P to start an offline refinement process deployed on the server side. Using the curated experience pool, we generate new model parameters that go beyond the limitations of the original parameter space exploration. We first train a VAE to learn the latent representation space of the model from P . On this basis, we train a diffusion model to sample and perform reverse restoration in this space, thereby generating a higher-quality global model and improving the performance of the global model through iterative refinement.

1) *LDM Application in Parameter Space*: Inspired by the powerful generative ability of the diffusion model, we transform the application of LDM from common image data to the weight parameters of neural networks. It is extremely difficult to directly apply generative models on model parameter spaces with millions of dimensions (curse of dimensionality). Since LDM operates in a latent space, it can better capture advanced features of data, resulting in relatively high sample quality, particularly for complex data structures [18]. We use the LDM to transfer the diffusion process from the high-dimensional

Algorithm 2: Offline Generative Refinement

Input: Experience pool P_0 ; Refinement iterations T_{refine} ; Number of generated models M .

Output: The final best model W_{best} .

```
1 Server executes:
2 Initialize generative model  $LDM$  (VAE + Diffusion);
3 for  $t = 1, 2, \dots, T_{\text{refine}}$  do
4   Calculate average performance  $\text{Avg}(P_{t-1})$ ;
5    $LDM \leftarrow \text{TrainLDM}(P_{t-1})$ ;
6   Generate a model set:
7    $\{W_{\text{gen},t}^1, W_{\text{gen},t}^2, \dots, W_{\text{gen},t}^M\} \leftarrow \text{Generate}(LDM)$ ;
8   for each generated model  $W_{\text{gen},t}^m$  do
9     Evaluate  $A(W_{\text{gen},t}^m)$ ;
10    Identify  $W_{\text{worst}} \leftarrow \arg \min_{W \in P_{t-1}} A(W)$ ;
11    if  $A(W_{\text{gen},t}^m) > \text{Avg}(P_{t-1})$  then
12      Update  $P_{t-1}$  according to Eq.(11);
13 Obtain updated experience pool  $P_t$ ;
14  $W_{\text{best}} \leftarrow \arg \max_{W \in P_{T_{\text{refine}}}} A(W)$ ;
15 Return  $W_{\text{best}}$ ;
```

parameter space to the low-dimensional latent space [19]. We first train a VAE, which contains an encoder E_ϕ and a decoder D_θ , to learn the bidirectional mapping between model parameters W and their low-dimensional latent variable representation z . The encoder compresses the model parameters into latent variables $z = E_\phi(W)$, and the decoder reconstructs the model parameters $\hat{W} = D_\theta(z)$ from the latent variables. The training goal of VAE is to minimize the following loss function,

$$L_{\text{VAE}}(\phi, \theta; W) = \mathbb{E}_{z \sim E_\phi(W)} \left[\|W - \hat{W}\|^2 \right] - \beta \cdot D_{\text{KL}}(z \parallel p(z)), \quad (9)$$

where $p(z) = \mathcal{N}(0, I)$ denotes the prior distribution of the latent variable, which is a standard normal distribution. After training the VAE, we apply its encoder E_ϕ to all models in the experience pool and obtain a set of latent variables $\{z^{(1)}, \dots, z^{(K)}\}$. Then, we train a diffusion model on this low-dimensional latent space. The diffusion model consists of two processes. In the forward process, starting from a true latent variable z_0 , gradually add noise through T steps to obtain a noise sequence $z_0, \dots, z_T$. Reverse process: Our goal is to learn a denoising network $\epsilon_\psi(z_t, t)$, which can predict the added noise ϵ given the noise version z_t and the time step t . Its training loss function (simplified form) is defined as

$$L_{\text{DM}}(\psi) = \mathbb{E}_{z_0 \sim z^{(i)}, \epsilon \sim \mathcal{N}(0, I)} [\|\epsilon - \epsilon_\psi(z_t, t)\|^2], \quad (10)$$

where z_t is obtained by adding noise ϵ to z_0 .

2) *Model Generation and Evolution:* After LDM training is completed, we start the T_{refine} round iterative refinement loop. In each refinement iteration t :

- **Generation.** We sample M independent noise vectors $Z_t \sim \mathcal{N}(0, I)$ from the standard normal distribution,

gradually denoise them through the reverse process, and obtain a set of generated latent variables $\mathbb{Z} = \{Z_{\text{gen},t}^1, Z_{\text{gen},t}^2, \dots, Z_{\text{gen},t}^M\}$.

- **Decoding.** For each $Z_{\text{gen},t}^i \in \mathbb{Z}$, we use the VAE decoder, map $Z_{\text{gen},t}^i$ back to the high-dimensional space to obtain the new model parameters $W_{\text{gen},t}^i = D_\theta(Z_{\text{gen},t}^i)$.
- **Evaluation and update.** In iteration t , we evaluate the performance of the generated model $W_{\text{gen},t}^i$ and update the experience pool through a competitive mechanism based on its performance. If $A(W_{\text{gen},t}^i) > \text{Avg}(P_t)$, execute the following formula

$$P_{t+1} = (P_t \setminus \{W_{\text{worst}}\}) \cup \{W_{\text{gen},t}^i\}, \quad (11)$$

where P_t is the experience pool updated in the t iteration round, storing tuples in the form of $(W_i, A(W_i))$, W_i represents the model i stored in P , and $A(W_i)$ is the performance result of the model tested on the proxy validation set on the server side. $W_{\text{gen},t}^i$ is the model i generated by the diffusion model in the t iteration round. $\text{Avg}(P_t) = \frac{\sum_{W_i \in P_t} A(W_i)}{|P_t|}$ represents the average accuracy of all models in the experience pool, used as a threshold for model replacement.

In the $t + 1$ iteration, the model $W_{\text{gen},t}^i$ generated by the diffusion model will be evaluated. If its performance is better than the average accuracy of the experience pool, the model with the worst performance in the pool, W_{worst} , will be eliminated and $W_{\text{gen},t}^i$ will enter the pool.

The pseudocode of the second stage is presented in Algorithm 2. The LDM is first trained on the experience pool P_{t-1} to learn the underlying distribution of high-quality model parameters (lines 1–5). During each refinement step, the model begins with Gaussian noise and performs a reverse diffusion process to generate latent representations, which are then decoded by the VAE into candidate models (lines 6–7). Candidates whose performance exceeds the average accuracy of the current pool replace the lowest-performing models, resulting in an updated pool P_t . The updated pool P is subsequently used to retrain the LDM (lines 8–13). After T_{refine} iterations, the model achieving the highest validation accuracy in the final pool is selected as the output (lines 14–15).

V. EXPERIMENTS

A. Experimental Setup

Implementation. We implement FedPDR using PyTorch and conduct all experiments on a single NVIDIA RTX 4090 GPU. In the simulation setting, we run 180 communication rounds with $E = 3$ local training epochs per round and a batch size of 64. The learning rate is selected from 0.01, 0.001 according to the specific dataset to ensure stable convergence. The generative refinement module consists of a VAE for parameter compression into a latent space and a Denoising Diffusion Probabilistic Model (DDPM) for modeling the latent parameter distribution. The detailed hyperparameter configurations of both the FL backbone and the generative module are summarized in Table I. Additional implementation and settings for the real-world scenario are provided in Section V-D.

TABLE I: Hyperparameters for LDM components.

Hyperparameter	DeepConv	ResFed-Net
<i>Generative Optimization</i>		
Batch Size	32	50
VAE Training Steps	800	500
Diff. Learning Rate	2×10^{-4}	1×10^{-4}
Total Diff. Steps	2,000	
VAE Learning Rate	2×10^{-4}	
Weight Decay	1×10^{-5}	
<i>Model Configuration</i>		
Latent Dimension (d)	512	1,024
Denoising CNN Chan.	[1, 64, 128, 256, 512, 256, 128, 64, 1]	
Denoising Kernel Size	7	
Diff. Timesteps (T)	1,000	
$\beta_{start}, \beta_{end}$	$(1 \times 10^{-4}, 0.02)$	
VAE Enc./Dec. Chan.	[64, 128, 256, 512, 256, 32]	

Tasks, Datasets, and Models. To comprehensively evaluate the effectiveness and robustness of FedPDR, we conduct experiments using four widely used benchmark datasets, including image recognition and sequence processing tasks.

- **Fashion-MNIST, EMNIST:** These tasks use *DeepConv*, a three-stage CNN with increasing channel depths. Each stage integrates BN, ReLU, and Max-pooling. A Global Average Pooling (GAP) layer condenses features into a 32-dimensional vector for final linear classification.
- **CIFAR-10:** This task employs *ResFed-Net*, built on Residual Blocks with skip connections. The architecture uses Layer Normalization (LN) and structured downsampling to extract semantic representations, concluding with an MLP classification head.
- **HAR:** This task utilizes *SenseConv-1D*, a lightweight 1D-CNN for 9-channel temporal data. It features a dual-stage extractor with ReLU and high-rate Dropout. Features are reduced via Max-pooling and a 100-unit dense layer to predict six activity classes.

Data Distribution and Participation. We partition the dataset for each client by using the Dirichlet distribution $Dir(\alpha)$. We set hyperparameters $\alpha \in \{0.2, 0.04\}$ to simulate different degrees of data heterogeneity. To evaluate the scalability and robustness of our framework, we conduct experiments under three different network sizes with the number of clients $N \in \{10, 20, 50\}$. For smaller network scales ($N = 10$ and $N = 20$), all clients participate in training in each round. For larger deployments ($N = 50$), we randomly select 60% of clients to participate in training in each round. We assume that the central server has a proxy dataset for evaluating the model. This is a common assumption in FL.

Baselines. We evaluate the performance of FedPDR against six mainstream and state-of-the-art FL paradigms, covering a spectrum from classical aggregation protocols to the most recent diffusion-based generative methods:

TABLE II: Comparison of global model accuracy (%).

Method	F-MNIST (DeepConv)				CIFAR-10 (ResFed-Net)			
	$Cl.=10$		$Cl.=20$		$Cl.=10$		$Cl.=20$	
	$\alpha=0.2$	$\alpha=0.04$	$\alpha=0.2$	$\alpha=0.04$	$\alpha=0.2$	$\alpha=0.04$	$\alpha=0.2$	$\alpha=0.04$
Centralized*	89.07				59.55			
FedAvg	84.33	69.18	85.59	60.48	48.86	42.69	50.41	32.07
FedProx	84.92	71.42	85.68	61.59	50.17	44.61	50.73	32.69
MOON	85.24	73.88	85.61	62.73	50.86	45.82	51.98	33.54
FedGKD	84.72	74.18	85.86	62.46	49.56	44.99	50.39	33.80
pFedGPA	84.93	69.70	85.88	61.31	49.38	43.89	52.00	36.43
FedP-Diff	84.37	69.23	85.59	60.85	48.86	42.75	50.41	32.36
FedAvg+PDR	86.94	78.18	85.80	64.75	52.44	48.26	53.91	35.73
FedProx+PDR	87.18	79.21	86.21	64.97	53.63	48.79	54.15	36.48
FedGKD+PDR	86.89	79.34	86.04	65.17	53.17	48.93	53.77	36.62

- **FedAvg** [13]: The seminal baseline in FL, which maintains a unified global model through coordinate-wise weighted averaging of local parameters.
- **FedProx** [9]: FedProx alleviates weight divergence caused by statistical heterogeneity by introducing a proximal regularization term.
- **MOON** [20]: A model-contrastive learning framework that corrects local training trajectories by maximizing the agreement between the representations of the current local model and the global model.
- **FedGKD** [21]: A knowledge-distillation-based method that utilizes a global model as a teacher to provide soft label guidance for client models, and standardizes local optimization through cross-model knowledge transfer.
- **pFedGPA** [22]: This method utilizes a diffusion model to learn the distribution of client parameters and reconstructs the model through parameter inversion. We collect the client models from the last 30 rounds to train the diffusion model and generate new models for evaluation.
- **FedP-diff** [19]: An advanced parameter generation paradigm that employs DDPM to synthesize high-quality model weights. We integrate this generative mechanism into the standard FL orchestration, where the server leverages the diffusion process to facilitate the synthesis and convergence of the global model parameters.

B. Performance Comparison

1) *Global Model Performance:* We conduct extensive experiments using our custom baseline methods and models across multiple datasets, ensuring that all baselines share the same model architectures and training configurations. The performance of the global model is assessed under varying degrees of data heterogeneity and different numbers of clients. The results, summarized in Table II, indicate that increasing data heterogeneity generally leads to a decline in performance across all algorithms. For example, on CIFAR-10, the accuracy of FedAvg drops from 59.55% to 32.07%.

Our proposed FedPDR, when integrated into existing methods, can significantly mitigate the adverse impact of data

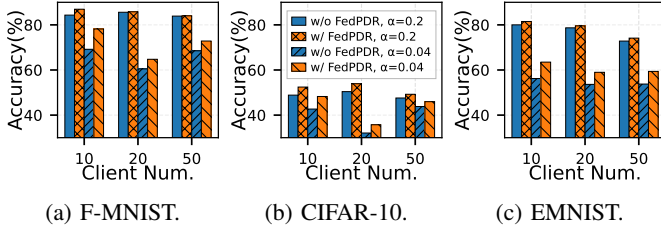


Fig. 4: The impact of different client numbers and different data heterogeneity on experimental results.

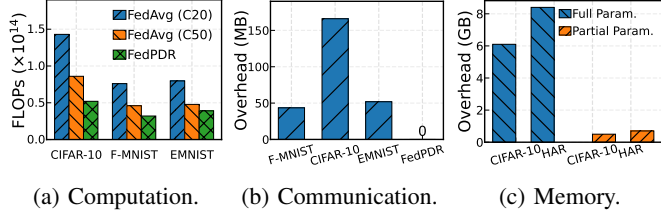


Fig. 5: Comparison results of communication overhead, computational overhead, and memory overhead.

heterogeneity on model accuracy. For example, when combined with FedAvg, we improve accuracy by 10.16% on Fashion-MNIST, and by 6.24% on CIFAR-10. This performance even exceeds some existing FL improvement methods. Furthermore, when integrated with FedProx, the results of certain tasks surpass those of all comparison methods, further demonstrating the effectiveness of our approach in addressing data heterogeneity. It is important to note that the FedPDR module can be seamlessly integrated into any existing FL method, enabling it to demonstrate superior performance when combined with more advanced FL algorithms.

2) *Impact of FL Configuration:* We conduct simulation experiments in FL environments of different scales. The experimental results are shown in Fig. 4. As the number of clients increases, the performance of the global model decreases across all datasets. In highly heterogeneous scenarios, the performance decline becomes more significant. In contrast, FedPDR demonstrates stronger robustness under the same settings and effectively mitigates the accuracy decline caused by data heterogeneity. These results indicate that even in large-scale, highly heterogeneous federated learning environments, FedPDR maintains stable performance.

3) *Computing and Communication Overhead:* We evaluate FedPDR across three dimensions: computational cost, communication overhead, and memory footprint. For computation (Fig. 5(a)), FedPDR significantly reduces computational costs compared to baselines; while traditional methods require training on extensive samples, our framework only involves sampling a limited number of models. For communication (Fig. 5(b)), FedPDR eliminates transmission overhead once initiated, as all clients no longer need to participate in FL training. For memory (Fig. 5(c)), LDM uses VAE to compress input parameters, so the memory overhead when generating parameters is controllable. In addition, compared to generating all parameters, generating some parameters can further

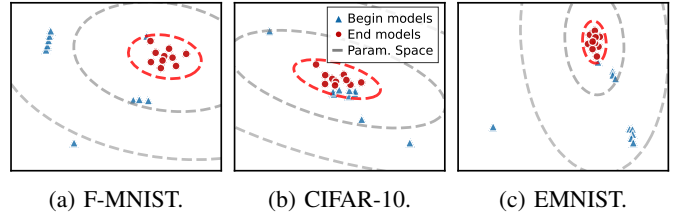


Fig. 6: Comparison of the distribution of experience pool models before and after refinement.

significantly reduce memory overhead.

4) *Model Parameter Distribution:* We analyze model parameter distributions before and after refinement by projecting global models into 2D using t-SNE. As shown in Fig. 6, models are initially scattered, indicating that linear aggregation alone cannot fully exploit the parameter space. After latent diffusion-based refinement, models converge into a compact cluster, demonstrating that our LDM captures the distribution of high-quality models. Instead of random sampling, LDM leverages historical elite models to construct a structured latent manifold, generating candidates near optimal regions. Each generation step balances exploration and exploitation, increasing the likelihood of superior parameters. This observation suggests that concentrated regions correspond to high-quality parameter configurations. The refined generator therefore maintains higher probability mass in these regions, which provides an intuitive explanation for why FedPDR surpasses conventional FL stagnation points. We also observed that as the number of iterations increased, the model gradually reached a stable solution space.

C. Hyperparameter sensitivity analysis

1) *Sensitivity Analysis of Memory Pool Size K :* To investigate the impact of the memory pool size K on the global model performance, we conduct sensitivity experiments across two datasets under varying degrees of data heterogeneity. The parameter K determines the number of historical model variants stored in the server-side experience pool.

As illustrated in Table III, we observe that FedPDR exhibits a robust performance trend across $K \in \{5, 10, 15, 20\}$. For the *ResFed-Net* on CIFAR-10, the accuracy peaks at $K = 10$ for both moderate ($\alpha = 0.2$) and extreme ($\alpha = 0.04$) heterogeneity. Similarly, for the *DeepConv* on Fashion-MNIST, $K = 10$ provides a superior or highly competitive accuracy compared to other configurations. These results suggest that a moderate pool size ($K = 10$) strikes an optimal balance: it provides sufficient historical knowledge to guide the diffusion model while avoiding the potential noise or stale model checkpoints introduced by an excessively large experience pool. Consequently, we fix $K = 10$ as the default setting for all subsequent experiments.

2) *Impact of Model Generation Quantity M :* The hyperparameter M denotes the number of candidate model variants synthesized by the diffusion process during each server-side iteration. As shown in Table IV, we investigate the influence of $M \in \{5, 10, 15, 20\}$ on the accuracy of the global model.

TABLE III: Sensitivity result of memory pool size (K).

Dataset	Setting (N, α)	Accuracy (%)			
		$K = 5$	$K = 10$	$K = 15$	$K = 20$
CIFAR-10 (ResFed-Net)	10, 0.2	54.15	54.19	54.00	54.07
	10, 0.04	47.68	48.28	47.94	48.23
F-MNIST (DeepConv)	10, 0.2	86.99	86.94	86.61	86.74
	10, 0.04	77.73	78.18	76.62	76.33

TABLE IV: Sensitivity result of model gen. quantity (M).

Dataset	Setting (N, α)	Accuracy (%)			
		$M = 5$	$M = 10$	$M = 15$	$M = 20$
CIFAR-10 (ResFed-Net)	10, 0.2	52.61	52.44	53.08	54.12
	10, 0.04	46.64	48.26	48.23	48.34
F-MNIST (DeepConv)	10, 0.2	86.66	86.48	86.51	86.79
	10, 0.04	76.65	76.73	77.29	77.83

Our observations indicate that under extreme data heterogeneity ($\alpha = 0.04$), increasing M leads to a consistent improvement in model accuracy. Specifically, for *DeepConv* on Fashion-MNIST, the accuracy rises from 76.65% ($M = 5$) to 77.83% ($M = 20$). This gain is attributed to the fact that a larger generation quantity expands the candidate space, thereby increasing the probability of synthesizing a model that effectively generalizes across diverse client distributions. Under moderate heterogeneity ($\alpha = 0.2$), the accuracy remains relatively stable, yet still achieves its peak at $M = 20$.

However, the increase in M inevitably introduces a trade-off in terms of server-side computational latency. In resource-constrained or latency-sensitive scenarios, a smaller M (e.g., $M = 5$ or 10) can still provide highly competitive performance. Therefore, M can be flexibly tuned based on the available server-side hardware capabilities and the specific convergence speed requirements of the target application. For subsequent evaluations, we choose $M = 10$ as the default value to maintain high processing efficiency.

D. Performance in Real-world Mobile Scenarios

To evaluate the performance of FedPDR in a real-world environment, we further deploy a realistic federated learning test platform. This physical system consists of eight edge nodes, including six Raspberry Pi 5 units, one NVIDIA Jetson Nano, and one NVIDIA Jetson Orin NX, as shown in Fig. 7. We develop a customized distributed FL training framework to manage these devices, utilizing socket programming to achieve reliable model transmission and command synchronization between the server and distributed hardware. This integrated setup enables us to rigorously verify the performance of algorithms in large-scale simulated environments and diverse physical hardware deployments. We conduct experiments on two datasets, HAR and Fashion-MNIST.

In order to reduce the memory overhead of parameter generation, we implement an efficient parameter generation

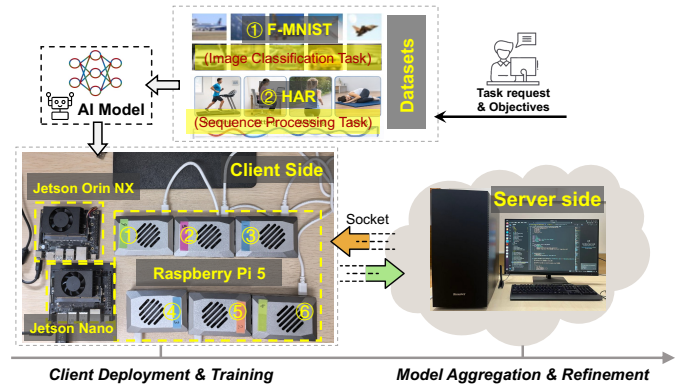
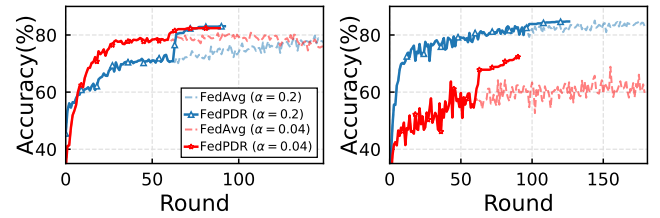


Fig. 7: Illustration of the real-world settings.



(a) HAR.

(b) F-MNIST.

Fig. 8: Experimental results in real environments.

strategy. Specifically, we only generate the parameters of the final FC layer of the model, rather than reconstructing the entire network architecture. This selective generation significantly reduces server memory usage and processing latency. For backbone generation, LDM follows the *DeepConv* and *ResFed-Net* architecture and hyperparameter settings, as detailed in Table I. This targeted configuration ensures a balance between model performance and system efficiency, allowing our framework to scale to larger models.

The performance of FedPDR under different heterogeneity settings in real-world scenarios is shown in Fig. 8. FedPDR effectively identifies convergence plateaus and triggers the post-optimization phase, leading to a significant performance leap. FedPDR achieves superior global accuracy while reducing communication rounds by up to 66%, demonstrating its robustness and efficiency in handling non-IID data.

E. Ablation Studies and System Analysis.

To comprehensively validate the effectiveness of our proposed framework, we conduct ablation studies focusing on two key components: the generative refinement module and the adaptive start-up mechanism.

1) *Effectiveness of Server-Side Generative Refinement*: We first evaluate the contribution of the post-training refinement strategy by comparing the performance of continuing standard FL training versus switching to our FedPDR refinement. To ensure fairness and consistency, we perform experiments with a fixed number of training epochs and initiate the offline refinement process at the 150th round. The experimental results indicate that, in the following 30 rounds, the server-side refinement process consistently improves the model's

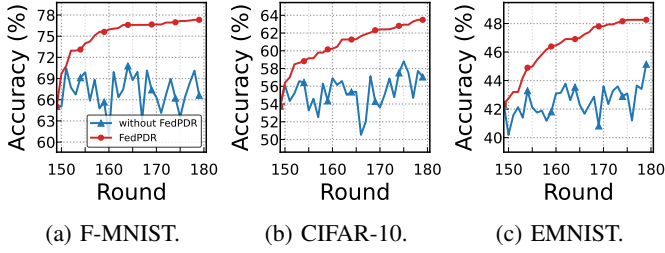


Fig. 9: Experimental results of post-plateau refinement.

accuracy, with the improvement surpassing that of traditional FL algorithms. The experimental results are shown in Fig. 9. Notably, after initiating the refinement process, the global model accuracy of traditional FL algorithms enters an oscillatory phase, making it challenging to achieve significant performance gains. In contrast, our method not only effectively mitigates the need for frequent communication between the client and server during this stage but also leads to a substantial improvement in global model accuracy within fewer iterations. These results underscore the advantages of our method in enhancing accuracy, reducing communication overhead, and accelerating convergence, thereby demonstrating its potential in addressing data heterogeneity and optimizing FL processes.

2) *Efficacy of the APD*: This section evaluates the effectiveness of APD in terms of autonomous recognition of performance plateaus compared to static fixed scheduling strategies. Fig. 10 shows the precision trajectory. Firstly, as shown by the gray dashed curve, the standard FedAvg falls into a performance plateau in the later stage due to noise caused by heterogeneity. On the contrary, both FedPDR variants have surpassed this bottleneck, validating the effectiveness of server-side post optimization methods. Secondly, this comparison highlights the efficiency advantage of APD. Unlike static strategies, APD detects diminishing returns using the signal-to-noise ratio and proactively triggers the post optimization process after detecting the performance plateau period. The adaptive strategy achieves convergence accuracy comparable to the static strategy, while significantly reducing communication time. This confirms that APD is a robust, resource-aware control mechanism suitable for restricted environments.

VI. RELATED WORK

A. FL with Data Heterogeneity

To stabilize optimization during local updates, FedProx [9] and FedDyn [23] introduce regularization terms, while FedAlign [24] adapts local learning to improve generalization. To improve aggregation quality and mitigate outlier or noisy client influence, hierarchical clustering [25], adaptive client selection [26], and Wasserstein distance-based weighting in FedImpr [27] have been developed. To reduce client drift via knowledge transfer, FedKT [28] and FedCAD [29] employ distillation and parameter sharing, while FedDC [30] introduces gradient correction. To further mitigate heterogeneity through generative modeling, CLIP2FL [31], [32], and [33] use diffusion models for data generation to bridge distribution gaps, while HyperDiffusion [34], MetaDiff [35], and other

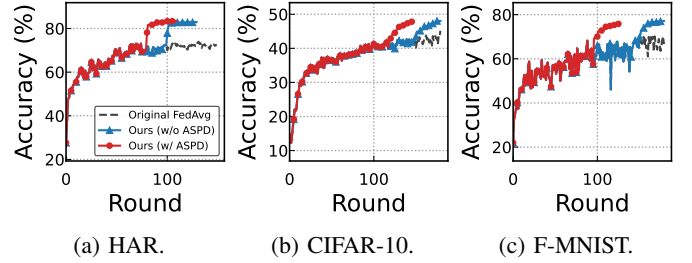


Fig. 10: Experimental results of APD algorithm ablation.

works [22], [36], [37] leverage them for parameter generation or guided local adaptation. In contrast to existing methods that are tightly integrated with iterative training and increase client-side computation, FedPDR operates solely as a server-side post-optimization strategy. By refining the global model after convergence, it eliminates further client involvement and additional local resource usage, offering a decoupled and efficient approach to overcome performance plateaus in FL.

B. Model Parameter Generation

Beyond classical gradient-based optimization, generating model parameters has emerged as a promising approach. To achieve efficient weight prediction and optimization, HyperTransformer [38] generates CNN weights from support samples, while [39] models parameters within attractor basins. To synthesize diverse and expressive parameters, diffusion-based techniques such as HyperDiffusion [34], MetaDiff [35], and conditional diffusion approaches [19], [40] have been proposed to enhance model diversity. Unlike prior parameter generation methods, which typically require collecting and training thousands of model weights, a process that is time-consuming and lacks direct optimization capabilities, FedPDR leverages only a small set of historical elite models. By incorporating an iterative generative refinement process, FedPDR performs post-training optimization on the global model, offering a novel approach to overcome the performance plateau in FL.

VII. CONCLUSION

In this paper, we proposed a federated learning post-optimization framework based on a latent diffusion model. The diffusion model is deployed on the server to learn the distribution of high-quality global model parameters, which is then used to generate refined model updates. By iteratively training on the generated parameter update experience pool, the server-side global model performance is continuously enhanced, mitigating the adverse effects of data heterogeneity. Moreover, because the server-side refinement does not require further client involvement, it eliminates the computational and communication overhead for resource-constrained edge devices. Experimental results on multiple benchmark datasets, model architectures, and real-world scenarios demonstrate the effectiveness and advantages of our approach. Future work will explore adaptive diffusion strategies and extend the framework to more heterogeneous and large-scale FL environments.

REFERENCES

- [1] K. S. Kaswan, J. S. Dhattewal, R. Yadav, K. Malik, and A. M. Tripathi, "Federated learning: Research, challenges, and future directions," in *Proceedings of the IEEE International Conference on Computing, Power and Communication Technologies (ICCCP)*, 2024, pp. 71–76.
- [2] K. Doshi and Y. Yilmaz, "Federated learning-based driver activity recognition for edge devices," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 3338–3346.
- [3] O. Aouedi, A. Sacco, L. U. Khan, D. C. Nguyen, and M. Guizani, "Federated learning for human activity recognition: Overview, advances, and challenges," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 7341–7367, 2024.
- [4] L. Xie, T. Luan, W. Cai, G. Yan, Z. Chen, N. Xi, Y. Fang, Q. Shen, Z. Wu, and J. Yuan, "dfmoe: Decentralized federated learning via mixture of experts for medical data analysis," in *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, 2025, pp. 10 203–10 213.
- [5] B. Zhang, Y. Mao, P. Zhang, H. Zheng, X. He, J. Kang, and J. Wu, "Adaptive federated learning for large models with scarce and non-iid data in cloud-edge networks," in *Proceedings of IEEE INFOCOM 2025-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2025, pp. 1–6.
- [6] Z. Lu, H. Pan, Y. Dai, X. Si, and Y. Zhang, "Federated learning with non-iid data: A survey," *IEEE Internet of Things Journal*, vol. 11, pp. 19 188–19 209, 2024.
- [7] J. Seo, F. O. Catak, and C. Rong, "Understanding federated learning from iid to non-iid dataset: An experimental study," *arXiv preprint arXiv:2502.00182*, 2025.
- [8] G. Yan, H. Wang, and J. Y. Li, "Seizing critical learning periods in federated learning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2022.
- [9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450.
- [10] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning (ICML)*. PMLR, 2020, pp. 5132–5143.
- [11] Y. Zhang, F. Liang, G. Yuan, M. Yang, C. Li, and X. Hu, "Fedpall: prototype-based adversarial and collaborative learning for federated learning with feature drift," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 3111–3120.
- [12] M. Yang, S. Su, B. Li, and X. Xue, "Exploring one-shot semi-supervised federated learning with pre-trained diffusion models," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 15, 2024, pp. 16 325–16 333.
- [13] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2016.
- [14] R. Lopez, P. Boyeau, N. Yosef, M. I. Jordan, and J. Regier, "Auto-encoding variational bayes," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [15] H. Vafaii, D. Galor, and J. Yates, "Poisson variational autoencoder," in *Proceedings of Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 44 871–44 906.
- [16] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proceedings of Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.
- [17] D. A. McAllester, "On the mathematics of diffusion models," *arXiv preprint arXiv:2301.11108*, 2023.
- [18] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10 674–10 685.
- [19] K. Wang, Z. Xu, Y. Zhou, Z. Zang, T. Darrell, Z. Liu, and Y. You, "Neural network diffusion," *arXiv preprint arXiv:2402.13144*, 2024.
- [20] Q. Li, B. He, and D. X. Song, "Model-contrastive federated learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10 708–10 717.
- [21] D. Yao, W. Pan, Y. Dai, Y. Wan, X. Ding, C. Yu, H. Jin, Z. Xu, and L. Sun, "Fedgkd: Toward heterogeneous federated learning via global knowledge distillation," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 3–17, 2023.
- [22] J. Lai, J. Li, J. Xu, Y. Wu, B. Tang, S. Chen, Y. Huang, W. Ding, and Y. Li, "pfedgpa: Diffusion-based generative parameter aggregation for personalized federated learning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 39, 2025, pp. 17 999–18 007.
- [23] C. Jin, X. Chen, Y. Gu, and Q. Li, "Feddyn: A dynamic and efficient federated distillation approach on recommender system," in *Proceedings of the IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 786–793.
- [24] W. Sun, R. Yan, R. Jin, R. Zhao, and Z. Chen, "Fedalign: Federated model alignment via data-free knowledge distillation for machine fault diagnosis," *IEEE Transactions on Instrumentation and Measurement*, vol. 73, pp. 1–12, 2024.
- [25] H. Chen and H. Vikalo, "Heterogeneity-guided client sampling: Towards fast and efficient non-iid federated learning," in *Proceedings of Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 65 525–65 561.
- [26] J. Zhang, J. Wang, Y. Li, F. Xin, F. Dong, J. Luo, and Z. Wu, "Addressing heterogeneity in federated learning with client selection via submodular optimization," *ACM Transactions on Sensor Networks*, vol. 20, no. 1, pp. 1–32, 2023.
- [27] Z. Tang, Y. Zhang, S. Shi, X. Tian, T. Liu, B. Han, and X. Chu, "Fed-impro: Measuring and improving client update in federated learning," *arXiv preprint arXiv:2402.07011*, 2024.
- [28] W. Mao, B. Yu, C. Zhang, A. K. Qin, and Y. Xie, "Fedkt: Federated learning with knowledge transfer for non-iid data," *Pattern Recognition*, vol. 159, p. 111143, 2024.
- [29] B. Zhu, S. Chang, G. Liang, H. Zhu, and J. Xu, "Fed-cad: Federated learning with correlation-aware adaptive local differential privacy," in *Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS)*, 2024, pp. 1–10.
- [30] L. Gao, H. Fu, L. Li, Y. Chen, M. Xu, and C. Xu, "Feddc: Federated learning with non-iid data via local drift decoupling and correction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10 102–10 111.
- [31] J. Shi, S. Zheng, X. Yin, Y. Lu, Y. Xie, and Y. Qu, "Clip-guided federated learning on heterogeneity and long-tailed data," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, 2024, pp. 14 955–14 963.
- [32] M. A. Hoefler, T. Mazouka, K. Mueller, and W. Samek, "Boosting federated learning with diffusion models for non-iid and imbalanced data," in *Proceedings of the IEEE International Conference on Big Data*, 2024, pp. 7790–7799.
- [33] M. Morafah, M. Reisser, B. Lin, and C. Louizos, "Stable diffusion-based data augmentation for federated learning with non-iid data," *arXiv preprint arXiv:2405.07925*, 2024.
- [34] Z. Erkoç, F. Ma, Q. Shan, M. Nießner, and A. Dai, "Hyperdiffusion: Generating implicit neural fields with weight-space diffusion," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 14 254–14 264.
- [35] B. Zhang and D. Yu, "Metadiff: Meta-learning with conditional diffusion for few-shot learning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2023.
- [36] Y. Wang, H. Wang, F. Zhang, and X. Li, "Fda-cdm: Data augmentation framework for personalized federated learning in non-iid scenarios," in *Proceedings of the International Conference on Electronics, Computers and Communication Technology*, 2023.
- [37] C. Xiao et al., "Confusion-resistant federated learning via diffusion-based data harmonization on non-iid data," in *Proceedings of Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [38] A. Zhmoginov, M. Sandler, and M. Vladymyrov, "Hypertransformer: Model generation for supervised and semi-supervised few-shot learning," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.
- [39] A. G. Wilson and P. Izmailov, "Bayesian deep learning and a probabilistic perspective of generalization," in *Proceedings of Annual Conference on Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 4697–4708.
- [40] X. Jin, K. Wang, D. Tang, W. Zhao, Y. Zhou, J. Tang, and Y. You, "Conditional lora parameter generation," *arXiv preprint arXiv:2408.01415*, 2024.